OmniScore Language Specification

Version: 1.1.0

Status: Normative

Maintainer: The OmniScore Working Group

License: MIT

Table of Contents

1. Introduction
2. Lexical Structure
3. Document Structure
4. Instrument Definitions (The Physics)
5. The Event Engine: Rhythm & Time
6. The Pitch Engine
7. Notational Attributes
8. The Tablature Engine
9. The Percussion Engine
10. Advanced Polyphony
11. Structure & Flow Control
12. The Lyric Engine
13. Layout Directives
14. Playback Control (The Synth Engine)
15. Macros & Variables
16. File Organization
17. Advanced Engraving Controls
18. Ornamentation & Lines
19. Microtonality
20. Visual Styling (The Theme Engine)
21. Advanced MIDI & Automation
22. Compiler Directives & Debugging
23. The Standard Library
24. Error Reference
25. Implementation Guidelines
26. Interoperability
27. Formal Grammar (EBNF)
28. Reference Example

1. Introduction

OmniScore is a declarative, domain-specific language (DSL) describing musical logic, notation, and performance data. Unlike XML-based formats which prioritize visual layout coordinates, OmniScore prioritizes **musical intent**, utilizing an inference engine to calculate layout at render-time.

1.1 Design Philosophy

The language adheres to three core principles:

1. **Inference Over Redundancy:** Attributes (duration, octave, velocity) persist until changed. This is known as "Sticky State" logic.
2. **Semantic Separation:** The definition of an instrument (its physics) is separated from the event data (the notes).
3. **Human Readability:** Source code should be intelligible to a musician without rendering software.

1.2 The Coordinate System

OmniScore maps music onto a logical grid:

- **X-Axis (Time):** Linear absolute time, segmented by Measure blocks.
- **Y-Axis (Source):** Distinct logical threads defined by Staff IDs.
- **Z-Axis (Polyphony):** Vertical layering within a single time/source coordinate (Voices).

1.3 The Compilation Pipeline

A compliant OmniScore compiler must follow these stages:

1. **Lexing/Parsing:** Tokenize the text and validate against the grammar.
2. **Context Building:** Process meta and def blocks to establish global physics (Time signatures, Instrument capabilities).
3. **Linearization:** Iterate through measure blocks.
    - Resolve "Sticky" attributes (fill in missing durations/octaves).
    - Calculate absolute tick positions for every event.
4. **Rendering:** Map the linearized data to the target output (SVG, MIDI, MusicXML).

2. Lexical Structure

2.1 Character Set & Encoding

Source files must be encoded in **UTF-8**.

- **Case Sensitivity:**
    - **Keywords** (def, measure, style) are case-insensitive.
    - **Note Names** (c4, C4) are case-insensitive.
    - **Identifiers** (vln, Gtr1) are case-sensitive.
    - **String Literals** ("Violin") are case-sensitive.

2.2 Whitespace & Formatting

Whitespace (Space U+0020, Tab U+0009, Line Feed U+000A) is generally insignificant and serves only as a token separator.

- **Indentation:** Two-space indentation is the recommended convention for readability but is not syntactically significant (unlike Python).
- **Line Breaks:** Valid statements may span multiple lines.

2.3 Comments

Comments are non-executable text segments used for documentation.

- **Line Comments:** Indicated by a double percentage sign %%. All text following %% on the same line is ignored.
  %% This is a comment
  c4:4 %% Inline comment

- **Block Comments:** Not currently supported to prevent parsing ambiguity.

2.4 Literals & Data Types

Type	Syntax	Examples	Description
Integer	[0-9]+	1, 120	Used for octaves, BPM, and repetition counts.
Float	[0-9]+\.[0-9]+	1.5	Used for precise duration multipliers.
String	".*"	"Violin I"	UI Labels and Metadata values.
Pitch	[a-g][#|b|x]*[0-9]	c4, f#5	Scientific Pitch Notation.
TabCoord	[0-9]+-[0-9]+	0-6, 12-2	Format: Fret-String.
Boolean	true	false	true
Array	[val, val]	[E2, A2]	Ordered lists.
Map	{ key: val }	{ k:0 }	Key-value pairs for configuration.

3. Document Structure

A valid OmniScore file consists of a single root object containing three optional but sequential sections: **Meta**, **Definitions**, and **Flow**.

3.1 The Root Block

The file must begin with the omniscore keyword.

```
omniscore
  %% 1. Metadata (Global)
  meta { ... }

  %% 2. Definitions (Setup)
  def ...

  %% 3. Flow (Logic)
  measure 1 ...
```

3.2 Metadata Scope

The meta block defines global properties.

Syntax: meta { key: value, ... }

Standard Keys:

- title (String): The work's title.
- composer (String): The author.
- tempo (Integer): Global beats per minute (BPM). Default 120.
- time (String): Initial time signature. Default 4/4.
- key (String): Initial key signature (e.g., C, F#m, Bb). Default C.
- swing (Integer): Swing percentage (0-100). Default 0.

4. Instrument Definitions (The Physics)

Before any music can be written, the "instruments" must be defined. This tells the parser how to interpret the data stream (e.g., interpreting 0 as a rest, a fret, or a drum hit).

4.1 The Definition Statement

Syntax: def [ID] [Label] [Attributes]

- **ID:** A unique alphanumeric identifier used to reference the staff in the Logic block (e.g.,

vln).
- **Label:** The display name for the staff (e.g., "Violin I").

4.2 Staff Styles

The style attribute determines the parsing engine for the staff.

4.2.1 Standard Style (style=standard)

The default engine for pitched instruments.

- **Input:** Pitch Literals (c4) or Rests (r).
- **Clef:** treble, bass, alto, tenor.
- **Transpose:** Integer (semitones). E.g., transpose=+2 for Bb Clarinet (written C sounds Bb).

```
def flt "Flute" style=standard clef=treble
def sax "Alto Sax" style=standard transpose=+9
```

4.2.2 Tablature Style (style=tab)

The engine for fretted instruments.

- **Input:** Tab Coordinates (fret-string).
- **Tuning:** Array of pitches representing open strings (low to high).
- **Capo:** Integer representing the fret position of a capo.

```
%% Standard Guitar
def gtr "Guitar" style=tab tuning=[E2, A2, D3, G3, B3, E4]

%% Bass Drop D
def bass "Bass" style=tab tuning=[D1, A1, D2, G2]
```

4.2.3 Grid Style (style=grid)

The engine for unpitched percussion or rhythmic triggers.

- **Input:** Mapped Characters (k, s, h).
- **Map:** A dictionary linking input characters to vertical staff positions (0 = bottom line).

```
def kit "Drum Set" style=grid map={
  k: 0,  %% Kick (Bottom line)
  s: 4,  %% Snare (Middle line)
  h: 8,  %% Hi-Hat (Top space)
  c: 10  %% Crash (Above staff)
}
```

4.3 Grouping & Bracketing

Staves can be visually grouped using the group block. Nested groups are permitted.

Syntax: group [Label] symbol=[brace|bracket|line] { ... }

```
group "Strings" symbol=bracket {
  def v1 "Violin I" style=standard
  def v2 "Violin II" style=standard
  def vla "Viola" style=standard clef=alto

  group "Low Strings" symbol=bracket {
    def vc "Cello" style=standard clef=bass
    def db "Bass" style=standard clef=bass transpose=-12
  }
}
```

5. The Event Engine: Rhythm & Time

The core of OmniScore's efficiency lies in its handling of time. Unlike formats that require explicit start/stop times for every event, OmniScore treats music as a linear stream of durations.

5.1 Duration Syntax

Duration is denoted by a colon followed by a value (:value).

- **Base Values:** Inverse of the note type (Whole=1, Half=2, Quarter=4, Eighth=8, etc.).
- **Dotted Notes:** Add . for each dot (adds 50% of current value).
- **Triplets/Tuplets:** Defined via ratios or shorthand.

Logic	Syntax	Musical Equivalent
Quarter	:4	♩
Eighth	:8	♪
Sixteenth	:16	♬
Dotted Quarter	:4.	♩.
Double Dotted	:4..	♩..
Whole	:1	𝅝

Breve	:0.5	𝅜

5.2 Sticky Attributes (Contextual Inference)

To reduce file size, OmniScore employs "Sticky State" logic. If a duration is omitted from an event, the parser infers it from the **immediately preceding event** in that staff.

Example:

```
%% Explicit (Verbose)
c4:4 d4:4 e4:4 f4:4

%% Implicit (Optimized)
c4:4 d4 e4 f4
```

Scope of Stickiness:

1. **Staff-Local:** Stickiness does not cross between staves.
2. **Measure-Crossing:** Stickiness **does** persist across bar lines (measures).
3. **Reset:** Stickiness is reset only by an explicit change or a new definition block.

5.3 Tuplets

Tuplets divide a duration into equal parts contrary to the prevailing meter.

Syntax: (events):ratio or explicit duration math.

Standard Syntax:

```
%% 3 notes in the time of 2 (Triplets)
(c4 d e):3/2

%% 5 notes in the time of 4 (Quintuplets)
(c4 d e f g):5/4
```

5.4 Rests

Rests use the literal r. They adhere to the same duration syntax and sticky logic as notes.

- **Standard Rest:** r:4
- **Multi-Measure Rest:** r:1 * 4 (Rest for 4 whole notes).

6. The Pitch Engine

This section details how frequency data is encoded for style=standard.

6.1 Scientific Pitch Notation

Pitch is defined by a Note Name (A-G), an Accidental, and an Octave Number.

- **Format:** [Step][Accidental][Octave]
- **Examples:** C4 (Middle C), F#5, Bb2.

Sticky Octaves:
Like duration, octaves are sticky. If an octave is omitted, the engine assumes the octave of the previous note.

- **Constraint:** OmniScore does *not* use "closest proximity" logic (like ABC notation). It strictly uses the last declared integer.
- **Example:** c4 e g becomes c4 e4 g4.

6.2 Accidentals

Accidentals are explicit.

- #: Sharp
- b: Flat
- x: Double Sharp
- bb: Double Flat
- n: Natural (Force natural)

Key Signature Interaction:
Notes are assumed to be natural unless modified by the key signature in the meta block. However, explicit accidentals in code always override the key signature visually and logically.

6.3 Chords

Polyphony within a single voice (chords) is denoted by square brackets [].

Syntax: [note note note]:duration

```
%% C Major Chord (Quarter note)
[c4 e4 g4]:4

%% Sticky Duration applies to the whole chord
[c4 e4 g4]:4 [f4 a4 c5]
```

6.4 Ties

Ties connect two notes of the same pitch to extend duration.
Syntax: ~ (tilde) appended to the first note.

```
c4:4~ c4:8  %% Ties a quarter to an eighth
```

7. Notational Attributes

Attributes decorate events with dynamics, articulations, and text. They are appended to the event using dot notation.

Syntax: event:duration.modifier.modifier

7.1 Dynamics

Standard MIDI dynamics are supported.

- pppp to ffff
- sfz (Sforzando)
- fp (Forte-piano)

Example: c4:4.ff

7.2 Articulations

Common rendering instructions.

- .stacc (Staccato)
- .acc (Accent)
- .ten (Tenuto)
- .fermata (Pause)
- .marc (Marcato)

Example: c4:4.stacc.acc

7.3 Text Instructions

Arbitrary text can be attached to events for display.
Syntax: event."Text"
c4:4."Sim." %% Displays "Sim." above the note

8. The Tablature Engine

This section details specific syntax for style=tab.

8.1 Coordinate Syntax

Tablature uses a fret-string coordinate system.

- **Fret:** Integer (0-24).
- **String:** Integer (1 = High E, 6 = Low E on standard guitar).

- **Example:** 0-6 (Open Low E string).

8.2 Guitar Techniques

Techniques are applied as dot modifiers.

- **Bends:** .bu(interval) and .bd(interval).
  - Interval can be half, full, 1.5, etc.
  - Example: 12-2:4.bu(full)
- **Slides:** .sl (Slide to next note) or .sl(from) (Slide from nowhere).
- **Legato:** .h (Hammer-on), .p (Pull-off), .t (Tap).
- **Harmonics:** .harm (Natural), .ph (Pinch).

8.3 Strums

Chords in tab are stacked in brackets []. Directional arrows can be added.

- **Downstroke:** .down
- **Upstroke:** .up

[0-6 2-5 2-4]:4.down

9. The Percussion Engine

This section details syntax for style=grid.

9.1 Map-Based Input

Percussion relies on the map defined in the header.

- Input is strictly limited to the keys defined in the map object.
- Attempting to use a non-mapped key results in a compile-time error.

9.2 Percussion Modifiers

Specific articulations for drums.

- .ghost (Parenthesized note)
- .flam (Grace note before hit)
- .drag (Double grace note)
- .roll (Tremolo/Buzz)

k:4 s:8.ghost s:8.acc

10. Advanced Polyphony

OmniScore supports multi-threaded logic within a single staff using Voice Groups.

10.1 Voice Syntax

Voices are enclosed in curly braces {}. Within the block, specific voice identifiers (v1, v2, v3, v4) separate the streams.

Syntax:

```
staff_id: {
  v1: events... |
  v2: events... |
}
```

- **v1:** Default "Up stem" voice.
- **v2:** Default "Down stem" voice.

10.2 Synchronization Rules

Voices within a group must sum to the same total duration for the measure or defined region.

- **Constraint:** If v1 contains 4 beats of music, v2 must also contain 4 beats (padding with rests if necessary).
- **Error Handling:** If durations mismatch, the compiler throws a VoiceSyncError and flags the measure.

11. Structure & Flow Control

Musical structure often relies on non-linear progression (repeats, jumps). OmniScore handles this via explicit tokens placed at measure boundaries or within the event stream.

11.1 Bar Lines & Repeats

The standard measure terminator | can be modified to indicate repeats.

- |: - Start Repeat
- :| - End Repeat
- :|: - Double Repeat (End prev, start next)
- || - Double Bar (Section End)
- |] - Final Bar Line (End of Piece)

Usage:

```
measure 1
  vln: c4 d e f :|  %% Repeats this measure once
```

11.2 Volta Brackets (Endings)

Voltas (1st and 2nd endings) are defined using bracket notation [N. ...] at the start of the staff content.

Syntax: [number. events ...]

```
measure 4
  vln: [1. c4 d e f :| ]  %% 1st Ending + Repeat sign

measure 5
  vln: [2. c4 g4 c5 r | ] %% 2nd Ending
```

11.3 Navigation (Jumps)

Jumps affect the playback cursor and render specific symbols.

- **segno**: Marks the Segno point (Sign).
- **coda**: Marks the Coda point.
- **D.S.**: Instruction Dal Segno.
- **D.C.**: Instruction Da Capo.
- **Fine**: End of piece.

Syntax: meta { jump: "D.S. al Coda" }

12. The Lyric Engine

Lyrics are handled as a parallel data stream mapped to a specific staff.

12.1 Syllable Mapping

Lyrics are defined using the lyric keyword followed by a string. The engine maps syllables to notes 1-to-1.

- **Separators:** Space (new word), Hyphen - (syllable within word).
- **Underscore _**: Extends the word (Melisma).

Syntax: staff_id.lyric: "Text string"

```
measure 1
  vox: c4:4 d4 e4 f4 |
  vox.lyric: "Hel- lo world _"
```

Result:

- c4 -> "Hel-"
- d4 -> "lo"
- e4 -> "world"

- f4 -> (Extension line)

12.2 Melismas & Hyphens

- **Hyphenation:** Hyphens in the text string render centered between notes.
- **Melisma:** Underscores indicate that a single syllable spans multiple notes.

13. Layout Directives

While OmniScore minimizes visual formatting, explicit breaks are sometimes required for engraving.

13.1 Break Commands

Break commands are inserted into the flow as metadata or standalone tokens.

- **break: system**: Forces a new line (system break).
- **break: page**: Forces a new page.

Usage:

```
measure 4
  ... music ...
  meta { break: system }
```

13.2 Spacing Adjustments

To adjust the visual density of a specific measure:

- **stretch: float**: A multiplier for the measure width (Default 1.0).

```
measure 1
  meta { stretch: 1.5 } %% Makes this measure 50% wider
```

14. Playback Control (The Synth Engine)

OmniScore is designed to drive audio synthesis. While style defines the notation, playback attributes define the sound.

14.1 Instrument Patches

Instruments are assigned using General MIDI names or specific SoundFont IDs via the patch attribute in the definition.

Syntax: def id ... patch="Instrument Name"

```
def pno "Piano" style=standard patch="Acoustic Grand Piano"
```

```
def syn "Lead"  style=standard patch="Sawtooth Wave"
```

14.2 Mixer Attributes

Volume and Panning can be set globally in the definition or modified dynamically in the flow.

- vol: Integer (0-127). Default 100.
- pan: Integer (-64 to 63). 0 is Center.

Dynamic Changes:

```
measure 1
  vln: c4:4
  meta { vln.vol: 80, vln.pan: -64 } %% Drop volume, pan hard left
```

14.3 Tempo Maps

Tempo changes affect playback speed without altering the notation time signature.

- **Immediate Change:** meta { tempo: 140 }
- **Ramps (Accel/Rit):** meta { tempo: [100, 140], curve: linear } over the duration of the measure.

15. Macros & Variables

To reduce redundancy in repetitive music, OmniScore supports text-substitution macros.

15.1 Defining Macros

Macros are defined in the header or def block using the macro keyword.

Syntax: macro Name = { events... }

```
macro RiffA = { c4:16 d eb f g f eb d }
```

15.2 Invoking Macros

Macros are expanded at compile-time by prefixing the name with $.

```
measure 1
  gtr: $RiffA $RiffA |
```

15.3 Transposing Macros

Macros can be transposed upon invocation using +N (semitones).

```
measure 2
  gtr: $RiffA+2 | %% Plays RiffA shifted up 2 semitones
```

16. File Organization

Large works can be split into multiple source files.

16.1 Imports

The import statement includes the contents of another OmniScore file at the current position.

Syntax: import "filepath"

```
omniscore
  import "config/orchestra_setup.omni"

  measure 1
    ...
```

17. Advanced Engraving Controls

While OmniScore's engine handles standard engraving rules automatically (autobeaming, stem direction), professional scores often require manual overrides.

17.1 Manual Beaming

By default, the engine groups beams based on the Time Signature (e.g., 4/4 groups by quarter note). Manual beaming is achieved via event attributes.

- **.bm**: Begins a beam.
- **.bme**: Ends a beam.
- **Constraint**: If a beam is started but not ended before a rest or barline, the compiler will auto-close it and issue a warning.

Syntax:

```
%% Force beam across the rest
c8.bm d8 r8 e8.bme
```

17.2 Stem Direction

Stem direction is calculated based on the note's position on the staff and voice layer. It can be

forced manually.

- **.up**: Force stem up.
- **.down**: Force stem down.
- **.auto**: Reset to calculated direction.

```
c5:4.down  %% Forces stem down despite being high on the staff
```

17.3 Grace Notes

Grace notes are ornamental notes with no rhythmic value in the measure's total count. They are defined using the :grace duration keyword.

- **Acciaccatura** (Slashed stem): note:grace.slash (Default).
- **Appoggiatura** (No slash, takes value): note:grace.noSlash.

Syntax:

```
%% A grace note leading into a whole note
d4:grace c4:1
```

18. Ornamentation & Lines

OmniScore supports a wide range of standard musical symbols used to decorate notes.

18.1 Standard Ornaments

Ornaments are attached as attributes.

- **.tr**: Trill.
- **.mordent**: Lower mordent.
- **.turn**: Standard turn.
- **.invTurn**: Inverted turn.
- **.prall**: Upper mordent (Pralltriller).

Parameters:
Some ornaments accept an interval or accidental in parentheses.
c4:2.tr(flat) indicates a trill to the flat upper neighbor.

18.2 Glissando & Portamento

Continuous pitch slides are defined as attributes on the *starting* note. The engine automatically draws the line to the *next* note in the stream.

- **.gliss**: Straight line (Glissando).
- **.port**: Curved line (Portamento).

```
c4:4.gliss c5:4  %% Draws line from C4 to C5
```

18.3 Arpeggiation

Indicates that a chord should be rolled.

- **.arp**: Standard wavy line.
- **.arp(up)**: Arrow up.
- **.arp(down)**: Arrow down.

[c4 e4 g4]:4.arp

18.4 Ottava Brackets

Octave shifts (8va, 8vb) are handled via meta regions or start/stop tokens.

Syntax:

- meta { ottava: "8va" } ... meta { ottava: "loco" }
- **Values:** 8va, 8vb, 15ma, 15mb, loco (normal).

19. Microtonality

OmniScore supports non-12TET (Equal Temperament) tuning systems native to the syntax.

19.1 Quarter-Tone Syntax

Scientific pitch notation is extended with quarter-tone accidentals.

- qs: Quarter Sharp (𝄲)
- qf: Quarter Flat (𝄳)
- tqs: Three-Quarter Sharp (♯)
- tqf: Three-Quarter Flat (♭)

Example:

```
c4 d4eq e4  %% C, D quarter-flat, E
```

19.2 Cent Deviations

For precise microtonal synthesis or retuning, pitch can be offset by cents using the + or - operator on the pitch itself.

Syntax: pitch[+/-cents]

```
c4+14:4   %% C4 tuned 14 cents sharp
a4-50:4   %% A4 tuned 50 cents flat (Quarter tone)
```

Note: If both explicit quarter-tone syntax (cqf) and cent deviation are used, they are additive.

20. Visual Styling (The Theme Engine)

OmniScore separates musical data from visual presentation. The visual layer is controlled via the theme metadata or granular overrides.

20.1 Theme Profiles

Global visual settings are applied via the theme key in the meta block.

- "standard": Traditional engraving (Bravura-style font).
- "jazz": Handwritten appearance (Petaluma-style font).
- "educational": Larger noteheads, color-coded pitch support.

Syntax:

```
meta { theme: "jazz" }
```

20.2 Color Syntax

For educational materials (e.g., Boomwhackers) or analysis, individual events can be colored using hex codes or CSS color names.

Syntax: event.color("HexOrName") or shorthand .colorName.

Supported Shorthands: .red, .blue, .green, .orange, .purple, .black.

```
%% Highlight the leading tone
b4:4.red c5:1.green
```

20.3 Notehead Overrides

The shape of the notehead can be altered to convey specific performance techniques or educational symbols.

Syntax: event.head("shape")

- "x": Cross (Spoken/Percussive)
- "diamond": Harmonic
- "triangle": Percussion
- "slash": Rhythmic notation (no pitch)

```
%% Spoken text rhythm
```

```
c4:4.head("x")
```

21. Advanced MIDI & Automation

For high-fidelity playback, OmniScore exposes raw MIDI control beyond simple dynamics.

21.1 Control Change (CC) Messages

Discrete MIDI CC messages are attached to events using the .cc() modifier.

Syntax: event.cc(ControllerNumber, Value)

- **ControllerNumber:** Integer (0-127).
- **Value:** Integer (0-127).

```
%% Modulation Wheel (CC 1) set to max
c4:1.cc(1, 127)
```

21.2 Automation Curves

To create smooth changes (ramps) over the duration of a note or region, use an array of values [start, end].

Syntax: event.cc(Number, [Start, End])

Interpolation: The engine linearly interpolates the values over the duration of the event.

```
%% Swell expression (CC 11) from 0 to 100 over a whole note
c4:1.cc(11, [0, 100])
```

21.3 Keyswitches

Keyswitches (silent notes used to trigger sample library articulations) can be defined in the Instrument Definition to keep the score clean.

Syntax: def ... keyswitch={ Label: NoteNumber }

```
def vln "Violin" style=standard keyswitch={
  arco: 24,    %% C0
  pizz: 25     %% C#0
}

measure 1
  vln: c4:4.pizz  %% Triggers MIDI Note 25 before playing C4
```

22. Compiler Directives & Debugging

These directives control how the OmniScore compiler processes the source text, useful for debugging complex scores.

22.1 Strict Mode

By default, OmniScore is "Lenient" (it attempts to auto-correct missing beam ends or weird durations). "Strict" mode forces the compiler to throw errors for any ambiguity.

Syntax: meta { strict: true }

Behaviors:

- Forces explicit beam termination.
- Disables "Sticky" attributes across bar lines (requires explicit restatement).
- Validates Voice Group durations strictly.

22.2 Conditional Compilation

Directives to include or exclude blocks based on the render target.

Syntax: if (target) { ... }

Targets: midi, score, tab.

```
measure 1
  %% Played in audio, hidden in PDF
  if (midi) {
    keys: c2:1
  }

  %% Visible in PDF, silent in audio
  if (score) {
    keys: c4:4.text("Ad lib.")
  }
```

23. The Standard Library

OmniScore includes a set of pre-defined constants and maps to ensure compatibility across different renderers and players.

23.1 Standard Clefs

Supported clefs for the clef attribute in definitions.

Clef ID	Description
treble	G Clef on 2nd line
bass	F Clef on 4th line
alto	C Clef on 3rd line
tenor	C Clef on 4th line
perc	Neutral (Percussion) Clef
tab	6-line Tablature Block

23.2 Standard Tunings

Pre-defined arrays for the tuning attribute in style=tab.

- guitar_std: [E2, A2, D3, G3, B3, E4]
- guitar_drop_d: [D2, A2, D3, G3, B3, E4]
- bass_std: [E1, A1, D2, G2]
- uke_std: [G4, C4, E4, A4] (High G)

Usage:

def gtr "Guitar" style=tab tuning=guitar_drop_d

23.3 GM Percussion Map

A default map is available for style=grid matching General MIDI mapping.

Usage: map=gm_standard

Mappings (Partial):

- k: Kick (MIDI 36)
- s: Snare (MIDI 38)
- h: Closed Hat (MIDI 42)
- ho: Open Hat (MIDI 46)
- c: Crash (MIDI 49)
- r: Ride (MIDI 51)
- t1: High Tom (MIDI 50)

- t2: Low Tom (MIDI 45)

24. Error Reference

The OmniScore compiler emits specific error codes to aid debugging.

24.1 Error Levels

- **Warning (W):** Compilation proceeds, but output may be unexpected.
- **Error (E):** Compilation halts for the specific measure; renderer inserts a placeholder.
- **Fatal (F):** Compilation halts immediately.

24.2 Common Error Codes

Code	Level	Name	Description
E101	Fatal	SYNTAX_ERR	Malformed token or illegal character.
E201	Fatal	DEF_MISSING	Staff ID used in measure but not defined in def.
E301	Error	VOICE_SYNC	Voice durations in a group do not match.
E302	Error	TIME_OVERFLOW	Event content exceeds the measure's Time Signature.
W401	Warning	BEAM_OPEN	Beam started but not ended; auto-closed.
W402	Warning	OCTAVE_JUMP	Large octave jump detected; check for missing octave digit.

25. Implementation Guidelines

This section provides a roadmap for developers building OmniScore compilers, parsers, or

renderers.

25.1 The State Machine

Parsers must maintain a robust state machine to handle the context-sensitive nature of "Sticky Attributes." The state object should track:

- **Cursor Position:** Current tick (absolute time).
- **Current Measure:** Index and local time signature.
- **Staff State:** * Last active duration.
  - Last active octave.
  - Active Key Signature.
  - Active Voice Layer.

Constraint: The state must be mutable within a measure but checkpointed at measure boundaries to allow for safe "Seek" operations during playback.

25.2 Resolution Order

To ensure deterministic output, compilers must resolve the logic tree in this specific order:

1. **Lexical Analysis:** Tokenize the stream.
2. **Macro Expansion:** Recursively resolve all $Macro tokens.
3. **Definition Registry:** Build the Instrument map from def blocks.
4. **Meta Processing:** Apply global settings (Title, Tempo).
5. **Linearization:** Iterate through measure blocks:
   - Apply "Sticky" defaults to incomplete events.
   - Calculate duration in ticks.
   - Assign absolute start times.
6. **Validation:** Check for TIME_OVERFLOW or VOICE_SYNC errors.
7. **Rendering:** Emit the final artifact.

26. Interoperability

OmniScore is designed to sit in the middle of the toolchain, easily converting to and from other standards.

26.1 MusicXML Mapping

When exporting to MusicXML 4.0:

- omniscore root maps to <score-partwise>.
- def definitions map to <score-part> elements in the <part-list>.
- measure blocks map to sequential <measure> elements.
- events map to <note> elements.
  - pitch -> <pitch>
  - duration -> <duration> (Calculated based on divisions)

- style=tab events map to <notation><technical><fret>/<string>.

26.2 MIDI Mapping

When exporting to Standard MIDI File (SMF):

- **Resolution:** Default to 480 PPQ (Pulses Per Quarter).
- **Tracks:** Each def becomes a MIDI Track.
- **Events:**
  - Note On: Start time = Event absolute tick.
  - Note Off: End time = Start time + Duration (adjusted for articulations).
    - .stacc = 50% duration.
    - .ten = 100% duration.
    - Default = 90% duration (to allow breath).

27. Formal Grammar (EBNF)

A simplified Extended Backus-Naur Form (EBNF) grammar to aid parser implementation.

```
Program        ::= 'omniscore' MetaBlock? DefBlock+ MeasureBlock+
MetaBlock      ::= 'meta' '{' KeyValuePair (',' KeyValuePair)* '}'
DefBlock       ::= 'def' Identifier StringLiteral Attribute*
MeasureBlock   ::= 'measure' MeasureRange MetaBlock? StaffLogic*
StaffLogic     ::= Identifier ':' (Event | BarLine | VoiceGroup)+
VoiceGroup     ::= '{' (Identifier ':' Event+)+ '}'
Event          ::= (Note | Chord | Rest) Duration? Modifier*
Note           ::= PitchLiteral | TabCoord | PercussionKey
Chord          ::= '[' Note+ ']'
Duration       ::= ':' Value Dot*
Modifier       ::= '.' Identifier ('(' Args ')')?
```

28. Reference Example

The following "Kitchen Sink" example demonstrates the integration of multiple engines (Standard, Tab, Percussion) in a single document.

```
omniscore
  meta { title: "OmniScore Reference Suite", tempo: 130, style: "Jazz" }

  %% 1. DEFINITIONS
  group "Rhythm Section" symbol=bracket {
    def sax "Tenor Sax" style=standard clef=treble transpose=-14
    def gtr "Guitar"    style=tab      tuning=guitar_std
    def drm "Drum Kit"  style=grid     map=gm_standard
```

```
}

%% 2. MACROS
macro LickA = { c5:16 d eb f g f eb d }

%% 3. LOGIC
measure 1
  meta { time: 4/4 }

  %% SAX: Sticky duration logic + Articulation
  sax: c4:4.accent e g c5.stacc |

  %% GUITAR: Tablature + Strumming
  gtr: [0-6 2-5 2-4]:4.down [0-6 2-5 2-4].up r:2 |

  %% DRUMS: Complex syncopation
  drm: k:8. s:16 k:8 s k s k s:4.roll |

measure 2
  %% SAX: Polyphonic split (Harmonizing itself)
  sax: {
    v1: $LickA c5:2.fermata |
    v2: g4:1.fermata        |
  }

  %% GUITAR: Pitch bend technique
  gtr: 12-2:2.bu(full) 12-2.bd(0) |

  %% DRUMS: Fill
  drm: s:16 s s s t1 t1 t2 t2 c:1 |
```